

Phần 1. Tổng quan Design Pattern (10%)

- **Design Pattern là gì?** Là những giải pháp tổng thể, đã được kiểm chứng cho các vấn đề thiết kế phổ biến trong kỹ thuật phần mềm. Nó không phải là một đoạn code cụ thể, mà là một khuôn mẫu (template) để giải quyết vấn đề trong các tình huống khác nhau.
- **Mục đích:** Cung cấp thuật ngữ chung cho developer giao tiếp, tăng khả năng tái sử dụng code, giảm thiểu rủi ro thiết kế và giúp phần mềm dễ bảo trì, linh hoạt hơn (giảm coupling, tăng cohesion).
- **Các thành phần cốt lõi của một Pattern:**
 - **Intent (Mục đích):** Trả lời câu hỏi "Pattern này làm gì? Vấn đề cốt lõi nó giải quyết là gì?".
 - **Problem (Vấn đề):** Ngữ cảnh cụ thể hoặc thách thức thiết kế khiến pattern này trở nên cần thiết.
 - **Solution (Giải pháp):** Cách cấu trúc các class/object (UML) và luồng tương tác để giải quyết vấn đề.
 - **Consequences (Hệ quả):** Chi phí, lợi ích, và sự đánh đổi (trade-offs) khi áp dụng pattern (ví dụ: code linh hoạt hơn nhưng số lượng class tăng lên).

Phần 2. Phân tích và phân biệt Pattern (40%)

1. Builder Pattern

Design Problem:

- **Vấn đề:** Khi một object (ví dụ: Hóa đơn, Hợp đồng) có quá nhiều thuộc tính, việc tạo ra nó thông qua Constructor sẽ dẫn đến tình trạng "Telescoping Constructor" (Constructor phình to với hàng tá tham số, rất dễ truyền nhầm thứ tự).
- **Khó khăn nếu không dùng:** Đọc code rất khó hiểu (không biết tham số `true/false` ở vị trí thứ 5 có ý nghĩa gì). Rất khó tạo ra các phiên bản object khác nhau (ví dụ: hợp đồng chỉ có cọc, hợp đồng không cọc) nếu không viết thêm vô số overloaded constructors.

Cấu trúc Pattern:

- **Class Diagram:** Bao gồm **Builder** (Interface), **ConcreteBuilder** (Triển khai các bước xây dựng), **Product** (Object phức tạp cần tạo), và **Director** (Quản lý thứ tự thực thi các bước của Builder).
- **Sequence Diagram:** Client gọi Director -> Director gọi các phương thức `buildPartA()`, `buildPartB()` trên ConcreteBuilder -> Client lấy kết quả qua `getResult()`.

So sánh: Builder vs Factory Method

Tiêu chí	Builder	Factory Method

Cách khởi tạo	Khởi tạo qua nhiều bước (Step-by-step).	Khởi tạo trong một bước duy nhất (Single-step).
Mức độ phức tạp của Object	Dành cho các object rất phức tạp , cấu tạo từ nhiều phần nhỏ.	Dành cho các object đơn giản hoặc cần che giấu logic chọn class con.
Sự tập trung	Tập trung vào <i>cách</i> object được lắp ráp ra sao.	Tập trung vào <i>class nào</i> sẽ được khởi tạo.

2. Prototype Pattern

Design Problem:

- **Vấn đề:** Việc tạo một object mới từ đầu (**new Keyword**) tốn quá nhiều chi phí (ví dụ: phải gọi API, query Database nặng) hoặc bạn muốn copy một object nhưng class của nó bị ẩn (private).
- **Khó khăn nếu không dùng:** Gây nghẽn hiệu năng (Performance bottleneck) khi phải liên tục query DB để tạo các object có cấu trúc tương tự nhau. Gây dính chặt (Coupling) giữa code tạo object và các class cụ thể.

Cấu trúc Pattern:

- **Class Diagram:** Bao gồm **Prototype** (Interface khai báo phương thức **clone()**) và **ConcretePrototype** (Class thực thi việc copy chính nó).

So sánh: Prototype vs Clone thông thường

Tiêu chí	Prototype Pattern	Clone thông thường (Native Clone)
Tính linh hoạt	Gọi qua Interface, Client không cần biết class thực sự của object là gì (Decoupling).	Dựa vào hàm clone() của ngôn ngữ (như Object.clone() trong Java), Client bị dính chặt vào class cụ thể.
Mức độ copy	Cho phép tùy chỉnh logic Deep Copy (copy toàn bộ object con bên trong) một cách chủ động và an toàn.	Thường mặc định là Shallow Copy (chỉ copy tham chiếu), dễ gây lỗi thay đổi dữ liệu chéo.

Phần 3. Áp dụng vào sản phẩm nhóm (Boarding House Management System) (30%)

Áp dụng 1: Hợp đồng Thuê nhà (Builder Pattern)

- **Hiện trạng (Before Design):**
 - Class `RentalContract` đang có quá nhiều thuộc tính: `tenantId`, `roomId`, `startDate`, `endDate`, `depositAmount`, `monthlyRent`, `electricityRate`, `waterRate`, `internetFee`, `specialTerms`.
 - **Vấn đề:** Khởi tạo hợp đồng rất dễ sai sót tham số. Khó mở rộng thêm phí dịch vụ mới (Coupling cao ở constructor).
- **Thiết kế sau khi áp dụng (After Design):**
 - Sử dụng `RentalContractBuilder`. Client chỉ cần gọi các hàm chain-method như `.withDeposit()`, `.withElectricityRate()` cho những thông tin cần thiết.
- **Lợi ích đạt được:**
 - **Tăng tính dễ đọc (Readability):** Code tự mô tả (Self-documenting).
 - **Tuân thủ OCP (Open-Closed Principle):** Dễ dàng thêm thuộc tính mới cho hợp đồng trong tương lai mà không làm vỡ các constructor cũ.

Áp dụng 2: Mẫu Cấu hình Phòng trọ mặc định (Prototype Pattern)

- **Hiện trạng (Before Design):**
 - Khi Landlord tạo một tòa nhà mới, họ phải nhập tay cấu hình cho hàng chục phòng giống hệt nhau (Diện tích 25m2, Giá 3.000.000đ, Phí điện 3.5k, Phí nước 80k/người, có điều hòa). Việc query DB để lấy các thông số mặc định này lặp đi lặp lại rất tốn tài nguyên.
- **Thiết kế sau khi áp dụng (After Design):**
 - Sử dụng `RoomProfileTemplate` implements `Cloneable`. Tạo sẵn một object mẫu (Prototype) duy nhất trong memory. Khi tạo phòng mới, chỉ cần `clone()` từ bản gốc và đổi `roomNumber`.
- **Lợi ích đạt được:**
 - **Tăng hiệu năng (Performance):** Bỏ qua chi phí gọi DB hoặc khởi tạo phức tạp.
 - **Giảm Coupling:** Logic copy dữ liệu nằm gọn trong chính class `RoomProfileTemplate`.

Phần 4. Demo Code (20%)

1. Demo Builder: Quản lý Hợp đồng (Rental Contract)

Java

// Product

```
public class RentalContract {  
    private String tenantName;  
    private String roomNumber;  
    private double monthlyRent;  
    private double depositAmount;  
    private String specialTerms; // Optional
```

// Private constructor, chỉ cho phép Builder tạo

```
private RentalContract(ContractBuilder builder) {
```

```

        this.tenantName = builder.tenantName;
        this.roomNumber = builder.roomNumber;
        this.monthlyRent = builder.monthlyRent;
        this.depositAmount = builder.depositAmount;
        this.specialTerms = builder.specialTerms;
    }

    @Override
    public String toString() {
        return "Contract [Room=" + roomNumber + ", Tenant=" + tenantName +
            ", Rent=" + monthlyRent + ", Deposit=" + depositAmount +
            ", Terms=" + specialTerms + "]";
    }

    // Static Builder Class
    public static class ContractBuilder {
        private String tenantName;
        private String roomNumber;
        private double monthlyRent;
        private double depositAmount = 0.0; // Default
        private String specialTerms = "None"; // Default

        public ContractBuilder(String tenantName, String roomNumber) {
            this.tenantName = tenantName;
            this.roomNumber = roomNumber;
        }

        public ContractBuilder setMonthlyRent(double rent) {
            this.monthlyRent = rent;
            return this;
        }

        public ContractBuilder setDepositAmount(double deposit) {
            this.depositAmount = deposit;
            return this;
        }

        public ContractBuilder setSpecialTerms(String terms) {
            this.specialTerms = terms;
            return this;
        }

        public RentalContract build() {
            return new RentalContract(this);
        }
    }
}

```

```
// Client Sử dụng
public class Main {
    public static void main(String[] args) {
        // Luồng thực thi: Tạo hợp đồng linh hoạt, không cần nhớ thứ tự tham số
        RentalContract contract = new RentalContract.ContractBuilder("Nguyen Van A", "101")
            .setMonthlyRent(3500000)
            .setDepositAmount(3500000)
            .setSpecialTerms("Không nuôi thú cưng")
            .build();
        System.out.println(contract);
    }
}
```

2. Demo Prototype: Mẫu Phòng trọ (Room Template)

Java

// Prototype Interface

```
public interface RoomPrototype {
    RoomPrototype cloneRoom();
}
```

// Concrete Prototype

```
public class StandardRoom implements RoomPrototype {
```

```
    private String roomType;
```

```
    private double price;
```

```
    private String defaultFacilities;
```

```
    private String roomNumber; // Thuộc tính duy nhất thay đổi
```

```
    public StandardRoom(String roomType, double price, String facilities) {
```

```
        // Giả lập tác vụ khởi tạo mất thời gian (VD: Query DB lấy cấu hình)
```

```
        try { Thread.sleep(2000); } catch (InterruptedException e) {}
```

```
        this.roomType = roomType;
```

```
        this.price = price;
```

```
        this.defaultFacilities = facilities;
```

```
    }
```

// Logic Deep/Shallow Copy được đóng gói tại đây

@Override

```
    public RoomPrototype cloneRoom() {
```

```
        StandardRoom cloned = new StandardRoom(this.roomType, this.price,
this.defaultFacilities);
```

```
        return cloned;
```

```
    }
```

```
    public void setRoomNumber(String roomNumber) {
```

```
        this.roomNumber = roomNumber;
```

```
    }
```

```

    public void display() {
        System.out.println("Phòng " + roomNumber + " | Loại: " + roomType + " | Giá: " + price
+ " | Tiện ích: " + defaultFacilities);
    }
}

```

// Client Sử dụng

```

public class Main {
    public static void main(String[] args) {
        System.out.println("Đang load cấu hình phòng tiêu chuẩn từ DB...");
        StandardRoom template = new StandardRoom("Phòng đơn", 3000000, "Điều hòa,
Nóng lạnh, Giường");

        System.out.println("Tạo các phòng thực tế từ Template...");
        // Chỉ mất vài mili-giây để nhân bản thay vì đợi khởi tạo từ đầu
        StandardRoom room101 = (StandardRoom) template.cloneRoom();
        room101.setRoomNumber("101");

        StandardRoom room102 = (StandardRoom) template.cloneRoom();
        room102.setRoomNumber("102");

        room101.display();
        room102.display();
    }
}

```